

WEB-BASED TRANSPORTATION MANAGEMENT SYSTEM

PROJECT PLAN

Revised & Audited Edition | v3.0

Prepared for: Internal Stakeholders & Development Team

Classification: Confidential

Date: June 2026

Project Manager & System Developer: Alemu Chamada

BSc in Computer Science and Engineering, Adama Science and Technology University (ASTU)

REAL-TIME TRACKING | SEAT RESERVATION | AUTOMATED PAYMENT PROCESSING

Table of Contents

1. Executive Summary	3
1.1 Project Overview	3
1.2 Business Goals	3
1.3 Expected Outcomes	3
2. Project Overview	4
2.1 Background	4
2.2 Objectives	4
2.3 Scope	4
2.4 Assumptions and Constraints	5
3. Requirements	6
3.1 Functional Requirements	6
3.2 Non-Functional Requirements	9
3.3 Business Rules	9
3A. Role Upgrade and Profile Completion Workflow	10
3A.1 Workflow Overview	10
3A.2 Supported Roles and Profile Requirements	10
3A.3 Profile Status State Machine	11
3A.4 End-to-End User Flow	11
3A.5 Business Rules	13
3A.6 API Endpoints	13
3A.7 Validation and Verification Requirements	14
3A.8 Acceptance Criteria	14
3A.9 Database Entities and Schema	15
3A.10 Frontend Behavior Specification	16
4. Solution Design	17
4.1 System Architecture	17
4.2 Booking & Payment State Machine	17
4.3 Concurrency Control	18
4.4 Webhook Processing	18
4.5 Database Design	18
4.6 Technology Stack	19
4.7 API Design Principles	19
4.8 Security Controls	20
4.9 Integrations	20
5. Project Organization	21
5.1 Stakeholders	21
5.2 Team Roles and Responsibilities	21
6. Work Breakdown Structure (WBS)	22
6.1 Major Phases	22
6.2 Tasks and Deliverables	22

7. Project Schedule	24
7.1 Milestones	24
7.2 Timeline	24
7.3 Dependencies	25
8. Development Approach	25
8.1 Methodology	25
8.2 Development Process	26
8.3 Quality Standards	26
9. Testing Strategy	27
9.1 Testing Types	27
9.2 Acceptance Criteria	27
10. Risk Management	29
10.1 Risk Register	29
11. Budget and Resources	30
11.1 Resource Requirements	30
11.2 Cost Estimates (Monthly Infrastructure)	30
12. Deployment Plan	31
12.1 Environment Strategy	31
12.2 CI/CD Pipeline	31
12.3 Zero-Downtime Deployment	32
12.4 Rollback Plan	32
12.5 Reliability & Disaster Recovery	32
13. Operations and Maintenance	33
13.1 Support Plan	33
13.2 Monitoring and Observability	33
14. Project Governance	35
14.1 Communication Plan	35
14.2 Change Management	35
14.3 Reporting Process	36
15. Appendices	36
Appendix A: System Architecture Diagram	36
Appendix B: Entity Relationship Diagram (ERD)	36
Appendix C: API Endpoint Reference	37
Appendix D: Background Job System Design	38
Appendix E: Phased Feature Delivery Plan	38
Appendix F: Glossary	39
Major Changes Summary (v2.0 → v3.0)	40

1. Executive Summary

1.1 Project Overview

The Web-Based Transportation Management System (TMS) is a full-stack digital platform designed to modernize urban and inter-city transportation operations. Built on a modular monolith architecture using Node.js, React.js, PostgreSQL, Redis, and Socket.IO, the system replaces fragmented manual workflows with a unified, real-time digital platform.

The platform serves five primary user roles: Passengers, Drivers, System/Operations Administrators, Traffic Authorities, Garage Managers, and Fuel Station Managers. Each role is granted a tailored experience through role-based access control (RBAC), ensuring operational clarity, data integrity, and appropriate security boundaries.

All users register initially as Passengers through OTP-based authentication. A System Administrator may subsequently promote a user to a specialized role. Upon promotion, the user is required to complete role-specific profile information before accessing advanced features. This Role Upgrade and Profile Completion Workflow is a core platform capability designed to be secure, auditable, and extensible for future role types.

Payment processing is fully automated via a supported payment gateway (Stripe or Chapa). No manual payment approval steps exist in this system. Booking confirmation is driven exclusively by payment gateway webhooks, verified server-side.

1.2 Business Goals

- Eliminate manual, paper-based ticketing and booking processes
- Ensure revenue integrity through automated, gateway-verified payment processing
- Provide real-time operational visibility to fleet managers and traffic authorities
- Deliver a maintainable platform designed to support future multi-city expansion
- Ensure regulatory compliance through structured audit trails and reporting
- Maximize asset utilization through intelligent capacity management and trip scheduling
- Support seamless, secure role promotion and profile completion for all specialized user types

1.3 Expected Outcomes

Metric	Expected Outcome
Overbooking Rate	Reduced to 0% via DB-level transactional seat locking
Payment Processing	Fully automated via payment gateway webhooks; no manual intervention
Booking Cycle Time	Reduced from hours (manual) to under 5 minutes (digital)
Real-Time Tracking	Continuous GPS updates delivered to passengers every 5-15 seconds
System Uptime Target	99.5%+ with staged rollout and rollback capability
Audit Coverage	100% of booking, payment, role change, and system actions logged immutably
Role Promotion Completion	Users promoted to specialized roles must complete role profile before feature access is granted

2. Project Overview

2.1 Background

Traditional transportation systems are hindered by manual seat allocation, absence of real-time visibility, fragmented payment flows, and poor stakeholder coordination. These issues result in overbooking, revenue loss, passenger dissatisfaction, and operational inefficiency. This project addresses these pain points by delivering a digitally integrated transportation management platform built for production deployment and operational maintainability.

2.2 Objectives

- Digitize and streamline end-to-end transportation operations
- Provide real-time vehicle tracking through WebSocket-based GPS broadcasting
- Implement concurrency-safe seat reservation with automatic expiration
- Ensure secure, fully automated payment processing via gateway webhooks
- Support trip lifecycle management from scheduling through completion
- Deliver an intuitive, role-specific UI for all user types, including a guided profile completion flow for newly promoted roles
- Establish a production-oriented infrastructure with monitoring, logging, and audit trails
- Implement a secure, extensible Role Upgrade and Profile Completion Workflow supporting future role expansion

2.3 Scope

Phase 1 — Core Production MVP (Weeks 1–22)

The following features constitute the initial production release:

- OTP-based multi-factor authentication with JWT session management
- Role-based access control (Passenger, Driver, System Administrator, Traffic Authority, Garage Manager, Fuel Station Manager)
- Role Upgrade and Profile Completion Workflow: promotion by administrator, mandatory profile completion gate, and verification for applicable roles
- Trip scheduling and lifecycle management
- Concurrency-safe seat reservation with automatic expiration
- Fully automated payment gateway integration (Stripe / Chapa) with webhook-based booking confirmation
- Real-time GPS tracking via Socket.IO
- Operational dashboard with revenue tracking, booking analytics, and role management interface
- Centralized audit logging for all critical actions including role promotions and profile completions
- Monitoring, alerting, and structured logging

- Background job system for booking expiration and retries
- Nearby bus and services discovery using Haversine distance calculation
- Traffic authority announcements system

Phase 2 — Enhanced Operations (Post-MVP)

- SMS/email/push notification delivery for booking confirmations, role promotion notices, and profile completion reminders
- Advanced ETA engine with real-time traffic data integration
- Enhanced reporting and operational analytics
- Richer payment reconciliation and dispute management tooling

Phase 3 — Growth Features (Future Release)

- Native mobile application (React Native) for Android and iOS
- Passenger reviews and driver ratings
- Traffic-aware route suggestions via third-party map API integration
- Multi-operator / multi-city support
- Additional role types supported through the extensible Role Upgrade framework

Explicitly Out of Scope (Phase 1)

- Native mobile applications (deferred to Phase 3)
- Custom machine learning routing models
- Real-time in-app chat and messaging
- Third-party map API integration (beyond static Haversine calculation)
- Multi-city or multi-operator support

2.4 Assumptions and Constraints

Type	Item	Detail
Assumption	GPS availability	Drivers carry GPS-enabled devices with stable internet connectivity
Assumption	Payment gateway	Stripe or Chapa payment gateway is accessible and configured for the target market
Assumption	Webhook reachability	The backend server is reachable by the payment gateway for webhook delivery
Assumption	Internet connectivity	Passengers and drivers have reliable internet access on supported devices
Assumption	Role promotion flow	Administrators verify eligibility of a user externally before initiating role promotion via the admin API
Constraint	Phased mobile rollout	Web application delivered first; native mobile app follows in Phase 3
Constraint	Single active trip	Each bus may have only one active trip at any time
Constraint	Payment channel	All payments processed exclusively through the integrated payment gateway; no manual payment channels
Constraint	Profile completion gate	A user promoted to a specialized role cannot access role-specific features until the mandatory profile is complete and, where required, administrator-verified

3. Requirements

3.1 Functional Requirements

Authentication & Identity

- Password + OTP two-factor registration and login flow. Users register with email, phone, and password. After password validation, an OTP is generated and printed to the terminal (Phase 2: delivered via SMS or email). Upon successful OTP verification, the system automatically assigns role = "passenger" — no role is selectable by the user during registration.
- OTP expiry: 5 minutes; maximum 3 verification attempts per OTP
- Rate limiting: 5 OTP requests per hour per user
- JWT-based stateless session authentication
- Secure logout with server-side token invalidation (token blacklist in Redis)
- Role assignment is system-controlled: after OTP verification, the backend automatically assigns role = "passenger" as the default; no role field is accepted from the client during registration
- System Administrator accounts are NOT created via public registration; they are provisioned exclusively by the developer during system initialization, or by an existing System Administrator via a secured admin operation
- Role upgrade (passenger → driver, traffic_authority, garage_manager, or fuel_station_manager; any role → system_admin) is performed exclusively by a System Administrator via PATCH /api/v1/admin/users/:id/role, after external verification/approval; no automatic promotion logic exists
- Upon role promotion, the system sets the user's profile_status to INCOMPLETE and records the promoted_at timestamp
- On the user's next login following promotion, the system detects INCOMPLETE profile status and returns a profile_completion_required flag in the JWT/login response; the frontend redirects the user to the mandatory profile completion flow before allowing access to any role-specific features
- Role-specific features and API endpoints remain inaccessible until profile_status = VERIFIED (or COMPLETE for roles not requiring admin verification)
- Role changes take effect immediately: the user's JWT payload is refreshed on their next authentication event; the frontend role-specific UI updates automatically upon token refresh
- Users MUST NOT modify their own role; role escalation attempts are rejected with HTTP 403 Forbidden; RBAC enforcement is applied at middleware level on all protected routes

3.1.1 Registration Workflow

The following sequence defines the exact steps executed by the system during user registration. All steps are server-enforced; client behaviour is not trusted for any security decision.

1. User submits registration form with the following fields:

- firstName (required, max 100 characters)
 - lastName (required, max 100 characters)
 - email (required, valid RFC 5322 format)
 - phone (required, valid E.164 format, e.g. +251911234567)
 - password (required, minimum 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character)
 - confirmPassword (required, must match password field; validated client-side and re-validated server-side)
2. Server queries the users table to check whether the submitted email or phone already exists.
 3. If email already exists in the users table: reject with HTTP 409 Conflict; error code EMAIL_TAKEN.
 4. If phone already exists in the users table: reject with HTTP 409 Conflict; error code PHONE_TAKEN.
 5. If both email and phone are unique: hash the password using bcrypt with a work factor of 12. The plaintext password is never stored, logged, or transmitted after this point.
 6. Save the new user record to the users table. The password_hash column stores the bcrypt hash. The role column is set to 'passenger' automatically by the database DEFAULT constraint — it is never accepted from the client.
 7. Generate a 6-digit numeric OTP. OTP is valid for 5 minutes. Maximum 3 verification attempts per OTP.
 8. Store the OTP record in the otp_codes table: user_id, contact (email or phone), hashed OTP code, expires_at (NOW() + 5 minutes), attempts = 0, purpose = 'registration'.
 9. Print the OTP to the server terminal (stdout) for development and testing purposes. In production, this step is replaced by SMS or email delivery (Phase 2).
 10. User submits the OTP via POST /api/v1/auth/verify-otp. Server retrieves the otp_codes record, checks the code hash, verifies the OTP has not expired, and increments the attempts counter. If attempts exceed 3, the OTP is invalidated.
 11. On successful OTP verification: set users.is_active = TRUE. The account is now active. The otp_codes record is marked as verified (verified_at = NOW()).

3.1.2 Login Workflow

The following sequence defines the exact steps executed by the system during user login. Password verification always precedes OTP generation. OTP is never generated for a failed password attempt.

12. User submits login credentials:
 - email OR phone (one required)
 - password (required)
13. Server queries the users table using the supplied email or phone.
14. If no matching user is found: return HTTP 401 Unauthorized with the generic message "Email or phone number or password is incorrect". The response does not reveal whether the identifier exists in the database.

15. If the user record is found but `is_active = FALSE`: return HTTP 401 Unauthorized with the generic message “Email or phone number or password is incorrect”. Active status is not revealed to the requester.
16. Validate the submitted password against the stored bcrypt hash using `bcrypt.compare()`. If the password does not match: return HTTP 401 Unauthorized with the generic message “Email or phone number or password is incorrect”. No OTP is generated.
17. Only after the password check passes:
 - Generate a 6-digit numeric OTP. OTP is valid for 5 minutes. Maximum 3 verification attempts.
 - Store the OTP record in the `otp_codes` table: `user_id`, `contact`, hashed OTP code, `expires_at`, `attempts = 0`, `purpose = 'login'`.
 - Print the OTP to the server terminal (`stdout`) for development and testing. In production, SMS or email delivery replaces this step (Phase 2).
18. User submits the OTP via POST `/api/v1/auth/verify-otp`. Server validates as described in the Registration Workflow (steps 10–11 above).
19. On successful OTP verification: generate a signed JWT. The JWT payload includes: `user_id`, `role`, `is_active`, `profile_completion_required` (true if `role_profiles.profile_status` is `INCOMPLETE` or `PENDING_VERIFICATION`), and `iat / exp` claims. JWT is signed with the server-side `JWT_SECRET` environment variable. Expiry: 24 hours (configurable).
20. Return the JWT in the response body. The frontend stores the JWT and redirects the user to their role-appropriate dashboard. If `profile_completion_required = true`, the frontend redirects to the Profile Completion Wizard instead.

3.1.3 Password Security Requirements

- Passwords are hashed using bcrypt with a minimum work factor of 12 before storage.
- Plaintext passwords are never stored, logged, cached, or transmitted after initial receipt.
- The `password_hash` column in the `users` table is the sole storage location for credential data.
- Password comparison is performed using a constant-time comparison function (`bcrypt.compare`) to prevent timing attacks.
- Login failure messages are always generic and never reveal whether the identifier exists or whether the password was the failing element.
- Brute-force protection: login endpoint rate-limited to 10 requests per minute per IP address. Accounts are locked for 15 minutes after 5 consecutive failed login attempts from the same user account.
- Password reset is out of scope for Phase 1 (no email/SMS delivery infrastructure). Administrators may reset passwords directly via a secured admin API endpoint: POST `/api/v1/admin/users/:id/reset-password`.
- `confirmPassword` field is validated server-side; a mismatch returns HTTP 422 Unprocessable Entity with field-level error detail.

Role Upgrade & Profile Completion

- System Administrator promotes a user by calling PATCH /api/v1/admin/users/:id/role with a valid target role
- On promotion, the system creates or updates the corresponding role_profiles record with profile_status = INCOMPLETE
- The promoted user is notified (Phase 2: via SMS/email) that their account has been upgraded and that profile completion is required
- On next login, GET /api/v1/auth/me returns profile_completion_required: true if profile_status = INCOMPLETE or PENDING_VERIFICATION
- The user submits required profile fields via PATCH /api/v1/profile/complete; the system validates all required fields per the role schema
- For roles requiring administrator verification (e.g., Driver: license verification; Garage Manager: facility registration), profile_status transitions to PENDING_VERIFICATION
- For roles not requiring administrator verification (e.g., Traffic Authority, Fuel Station Manager), profile_status transitions to COMPLETE upon successful submission
- System Administrator reviews and verifies the submitted profile via PATCH /api/v1/admin/users:id/profile/verify (approve or reject)
- Upon approval, profile_status transitions to VERIFIED; the user gains full access to role-specific features
- Upon rejection, profile_status reverts to INCOMPLETE; the user is notified with the rejection reason and must resubmit
- All profile status transitions are recorded in the audit_logs table
- The profile completion framework is designed for extensibility: new roles can be added by defining their required fields in the role_profile_schemas configuration without code changes to the workflow engine

Booking System

- Passenger initiates booking; system creates a booking record in RESERVED state
- Seat lock enforced via PostgreSQL transaction-level row lock (SELECT ... FOR UPDATE)
- Redis TTL lock (lock:trip:{id}:seat:{n}) used as optional pre-validation step before DB transaction
- Reservation auto-expires after 15 minutes if payment session is not completed
- Background job releases seat lock and transitions booking to EXPIRED on timeout
- Upon reservation creation, backend initializes a payment session with the payment gateway
- Booking state machine: RESERVED → PAYMENT_PENDING → CONFIRMED | FAILED | EXPIRED
- Payment gateway webhook triggers booking confirmation; frontend responses are never trusted for payment status
- Duplicate webhook events handled idempotently via idempotency key stored in Redis (24-hour TTL)
- Failed payments transition booking to FAILED state; seat is released

- Idempotency tokens prevent duplicate bookings on network retry

Capacity Management

- Seat states: Available, Reserved, Confirmed
- Constraint enforced at DB level: $\text{Reserved} + \text{Confirmed} \leq \text{Total Capacity}$
- No new booking allowed when capacity is fully reserved or confirmed

Real-Time GPS Tracking

- Driver emits GPS coordinates via Socket.IO every 5–15 seconds
- Server stores only the latest location per bus (single-row upsert)
- Location broadcasts to trip-specific Socket.IO rooms
- Passenger subscribes to trip room upon booking confirmation
- Fallback REST endpoint: POST /location available for degraded connectivity scenarios

Nearby Bus & Services Discovery

- Passenger location captured via browser Geolocation API
- Distance computed server-side using the Haversine formula
- Returns all active buses within a configurable search radius
- Nearby garages and fuel stations served from a static, extendable dataset

Trip Management

- Trip scheduling: start time, route, capacity, assigned bus and driver
- One active trip per bus enforced at both application and database level
- Driver-controlled trip lifecycle: scheduled → active → completed
- ETA calculated as $\text{distance} \div \text{configurable average speed}$; updates dynamically

Automated Payment Processing

- Passenger is redirected to payment gateway checkout upon reservation creation
- Backend never confirms payment based on frontend response
- Payment gateway sends webhook to POST /payments/webhook upon payment completion
- Backend verifies webhook signature using gateway-provided HMAC secret
- Verified successful webhook transitions booking to CONFIRMED and notifies passenger
- Verified failed webhook transitions booking to FAILED and releases the seat
- Duplicate webhook events are safely deduplicated using stored idempotency keys
- System Administrators may process refunds and manage disputes via gateway dashboard or admin API

Admin Operations Dashboard

- Revenue tracking: total, daily, weekly, monthly
- Booking statistics: confirmed, expired, failed, cancelled ratios
- Failed payment volume and retry rates
- Driver trip activity and fleet utilization reports
- Access to paginated, immutable audit log
- User management interface: list all users with role, profile_status, and promoted_at; initiate role promotions; review and verify/reject pending role profiles

Traffic Authority Content

- Create, publish, and manage traffic announcements
- User comment and interaction on posts

3.2 Non-Functional Requirements

Category	Requirement	Target / Mechanism
Performance	API response time	< 300ms for standard endpoints under normal load
Performance	WebSocket latency	GPS update propagation < 200ms end-to-end
Performance	Profile completion flow	Profile submission and status update < 500ms
Scalability	Stateless backend	No server-side session state; all state persisted in DB or Redis
Scalability	Redis caching layer	OTPs, seat locks, pub/sub, and session invalidation cache in Redis
Security	Rate limiting	Login: 10 req/min; Booking: 30 req/min; OTP: 5/hr/user; Role promotion: 20 req/min admin-only
Security	Input validation	Zod/Joi schemas on all endpoints; strict type coercion; role_profile fields validated against per-role schema
Security	HTTPS enforcement	TLS required on all environments except localhost development
Security	Secrets management	All credentials stored in environment variables or a secrets manager; never committed to source control
Reliability	Uptime target	99.5% in production environment
Reliability	Retry mechanism	Failed webhook deliveries and notifications retried with exponential backoff (up to 3 attempts)

Reliability	Health checks	Liveness and readiness endpoints at /health; integrated with monitoring alerts
Usability	Responsive UI	Mobile-first responsive design across all screen sizes
Usability	Profile completion UX	Profile completion wizard renders within 2 seconds; field errors displayed inline; partial progress saved to prevent data loss on navigation
Auditability	Audit logging	All booking, payment, role promotion, profile status change, and system actions immutably logged with actor, timestamp, entity, and metadata
Extensibility	Role framework	New role types and their required profile fields configurable without changes to the workflow engine code

3.3 Business Rules

Booking Rules

- A passenger may not hold more than one RESERVED or PAYMENT_PENDING booking per trip simultaneously
- A seat confirmed by successful payment cannot be re-allocated without an authorized cancellation or refund
- A driver may be assigned to only one active trip at any given time
- Booking confirmation is triggered exclusively by a verified payment gateway webhook
- System Administrators do not manually approve or reject individual bookings

Role & Profile Rules

- All users register as Passengers; no role may be selected or specified during registration
- Role promotion is performed exclusively by a System Administrator after external eligibility verification
- A user who has been promoted to a specialized role has profile_status = INCOMPLETE until the mandatory profile form is submitted
- A user with profile_status = INCOMPLETE or PENDING_VERIFICATION may not access any role-specific features or API endpoints beyond their Passenger capabilities
- Profile completion is mandatory and enforced server-side; frontend bypasses are rejected at middleware
- Roles requiring administrator verification (e.g., Driver) must have profile_status = VERIFIED before specialized access is granted
- Roles not requiring administrator verification (e.g., Traffic Authority, Fuel Station Manager) achieve COMPLETE status immediately upon successful profile submission
- A System Administrator may reject a submitted profile, setting it back to INCOMPLETE with a mandatory rejection reason

- A user may be promoted again to a different role; prior role_profile records are retained in audit history
- System Administrators do not complete a role-specific profile; their operational access is granted at account creation
- The role promotion system must support addition of new role types without changes to the core workflow engine

Content & Access Rules

- Traffic posts may only be created by users with the traffic_authority role AND profile_status = COMPLETE or VERIFIED
- Booking expiration is enforced by the background job system, not by client-side timers
- All payment decisions are handled automatically by the payment gateway; human administrator intervention is limited to refunds and dispute resolution

3A. Role Upgrade and Profile Completion Workflow

This section defines the complete specification of the Role Upgrade and Profile Completion Workflow — a core platform capability that governs how users transition from the default Passenger role to a specialized operational role, and how they must complete and verify their role-specific profile before accessing expanded system features.

3A.1 Workflow Overview

All users enter the system as Passengers. A System Administrator may promote a user to a specialized role at any time. Promotion alone does not grant feature access; the user must complete a mandatory role-specific profile, and in some cases, await administrator verification, before role-specific capabilities are unlocked.

The workflow is designed to be:

- **Secure:** Role promotion is admin-only; profile access gates are enforced server-side at middleware.
- **Auditable:** Every role change and profile status transition is recorded in the immutable audit_logs table.
- **Scalable:** New roles are supported by adding a schema definition; no workflow engine changes are required.
- **User-Friendly:** The profile completion wizard provides inline validation, partial progress saving, and clear guidance on required fields.

3A.2 Supported Roles and Profile Requirements

Role	Required Profile Fields	Admin Verification?	Final Profile Status
passenger	Name, phone number (collected at registration)	No	N/A — default role
driver	Full name, driver license number, license expiry date, vehicle type, profile photo	Yes	VERIFIED
traffic_authority	Full name, authority organization, badge/ID number, jurisdiction area	No	COMPLETE
garage_manager	Full name, garage name, garage address, business registration number, contact number	Yes	VERIFIED
fuel_station_manager	Full name, station name, station address, operator license number, contact number	No	COMPLETE

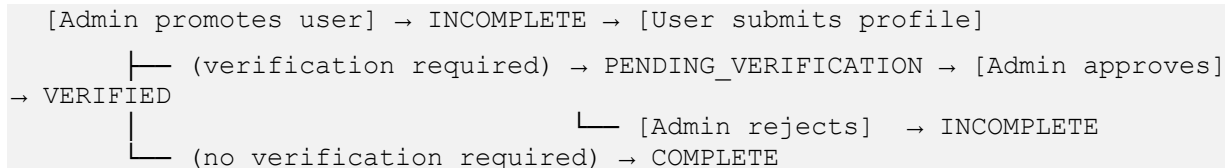
system_admin	Provisioned directly; no role-profile form required	No	N/A — provisioned by developer/existing admin
--------------	---	----	---

3A.3 Profile Status State Machine

The profile_status field on the role_profiles table follows a strict state machine enforced server-side:

Status	Meaning	Trigger	Next Status
INCOMPLETE	Profile has not been submitted. Role-specific features are locked.	Admin promotes user to role	PENDING_VERIFICATION or COMPLETE
PENDING_VERIFICATION	Profile submitted; awaiting admin review (verification-required roles only).	User submits profile form	VERIFIED or INCOMPLETE (on rejection)
COMPLETE	Profile submitted and accepted (no admin verification required).	Successful profile submission (non-verification roles)	Terminal
VERIFIED	Profile submitted and approved by System Administrator.	Admin approves profile via admin API	Terminal

Text Diagram — Profile Status State Machine:



3A.4 End-to-End User Flow

Step 1: Registration (Passenger)

21. User provides phone number or email on the registration screen (no role selector present)
22. System sends an OTP; user verifies within 5 minutes
23. Backend creates user record with role = 'passenger'; no role_profiles record is created at this stage

24. JWT issued with role = 'passenger' and profile_complete = true

Step 2: Administrator Promotion

25. System Administrator navigates to the User Management section of the admin dashboard
26. Admin locates the user and selects a target role from the available promotion options
27. Admin calls PATCH /api/v1/admin/users/:id/role with body { role: 'driver' } (example)
28. Backend validates: admin is authenticated, target role is valid, user is not self-modifying
29. Backend updates users.role and creates a role_profiles record: { user_id, role, profile_status: 'INCOMPLETE', promoted_at: now() }
30. Audit log entry created: { action: 'ROLE_PROMOTED', actor_id, target_user_id, previous_role, new_role, timestamp }
31. (Phase 2) Notification job enqueued to inform user of promotion

Step 3: User Login After Promotion

32. User initiates login via OTP flow
33. After OTP verification, backend checks role_profiles for the user's current role
34. If profile_status = INCOMPLETE or PENDING_VERIFICATION, the JWT response includes: { profile_completion_required: true, incomplete_role: 'driver' }
35. Frontend detects the flag and redirects user to the Profile Completion Wizard
36. User cannot access any role-specific navigation, routes, or API endpoints until profile_status transitions to COMPLETE or VERIFIED

Step 4: Profile Completion

37. Frontend renders the Profile Completion Wizard with fields dynamically determined by the user's current role schema
38. User fills in required fields; inline validation is applied in real-time
39. Partial progress is auto-saved to localStorage / sessionStorage (frontend) and optionally staged server-side (role_profiles.draft_data)
40. User submits the completed form via PATCH /api/v1/profile/complete
41. Backend validates all required fields against the role's profile schema
42. If validation fails: HTTP 422 Unprocessable Entity with field-level error messages
43. If validation passes:
 - For verification-required roles: profile_status → PENDING_VERIFICATION; audit log entry created; admin notified
 - For non-verification roles: profile_status → COMPLETE; audit log entry created; user gains access immediately

Step 5: Administrator Verification (for Applicable Roles)

44. System Administrator reviews pending profiles via GET /api/v1/admin/users/pending-profiles
45. Admin reviews submitted documents and profile data
46. Admin calls PATCH /api/v1/admin/users/:id/profile/verify with { decision: 'approved' } or { decision: 'rejected', reason: '...' }
47. If approved: profile_status → VERIFIED; audit log entry created; user notified; user gains full role access
48. If rejected: profile_status → INCOMPLETE; rejection_reason recorded; audit log entry created; user notified and prompted to resubmit

Step 6: Full Role Access

49. User logs in; JWT reflects current role and profile_complete = true
50. Frontend renders role-specific dashboard and navigation without any completion gate
51. All role-specific API endpoints are accessible

3A.5 Business Rules

- BR-RP-01: All users must register as Passengers; no alternative entry role is permitted
- BR-RP-02: Role promotion is exclusive to System Administrators; self-promotion is blocked with HTTP 403
- BR-RP-03: Upon promotion, a role_profiles record is immediately created with profile_status = INCOMPLETE
- BR-RP-04: A user with profile_status = INCOMPLETE or PENDING_VERIFICATION is restricted to Passenger-level capabilities only
- BR-RP-05: Profile completion is enforced server-side; RBAC middleware checks profile_status before granting access to specialized endpoints
- BR-RP-06: For verification-required roles, the profile_status must reach VERIFIED (not merely COMPLETE) before feature access is granted
- BR-RP-07: A rejected profile must include a machine-readable rejection_reason field in the admin request body
- BR-RP-08: Every profile status transition must be recorded in audit_logs with the full before/after state
- BR-RP-09: A user who has been promoted to a new role retains prior role_profile history; records are never deleted
- BR-RP-10: The system must support adding new role types by defining a schema entry; no changes to the workflow engine are required
- BR-RP-11: System Administrator accounts are never subject to the profile completion flow
- BR-RP-12: A user may hold only one active role at any time; multiple simultaneous roles are not supported in Phase 1

3A.6 API Endpoints

Method	Endpoint	Auth	Description
PATCH	/api/v1/admin/users/:id/role	system_admin	Promote or change a user's role. Body: { role }. Validates role enum, blocks self-change, logs audit entry, creates role_profiles record.
GET	/api/v1/auth/me	Authenticated	Returns current user profile including profile_completion_required flag and incomplete_role if applicable.
PATCH	/api/v1/profile/complete	Authenticated	Submit role-specific profile fields. Validates against role schema. Transitions profile_status to PENDING_VERIFICATION or COMPLETE.
GET	/api/v1/profile/schema	Authenticated	Returns the required fields schema for the user's current role. Used by the frontend to render the profile wizard dynamically.
GET	/api/v1/admin/users/pending-profiles	system_admin	Lists all users with profile_status = PENDING_VERIFICATION, including submitted profile data.
PATCH	/api/v1/admin/users/:id/profile/verify	system_admin	Approve or reject a submitted role profile. Body: { decision: 'approved' 'rejected', reason? }. Transitions profile_status and logs audit entry.
GET	/api/v1/admin/users	system_admin	Paginated list of all users with current role, profile_status, and promoted_at.

3A.7 Validation and Verification Requirements

Profile Submission Validation

- All required fields defined in the role schema must be present; missing fields return HTTP 422 with a field-level error map
- Text fields: minimum and maximum length constraints applied per field; no script injection characters accepted
- License / registration numbers: format validated against configurable regex patterns per role

- Dates (e.g., license expiry): must be in the future at time of submission; must be in ISO 8601 format
- File uploads (profile photo, document scans — Phase 2): validated for file type (JPEG/PNG/PDF), maximum size (5 MB), and virus-scanned before storage
- Duplicate detection: unique license numbers and registration numbers enforced at DB level (unique index on role_profiles.license_number, registration_number)

Administrator Verification Requirements

- Admins must supply a non-empty rejection_reason when rejecting a profile (HTTP 400 if omitted)
- Verification decision is recorded with verifier_admin_id, decision, reason (if rejected), and verified_at timestamp
- Admins may not verify their own profile or a profile they promoted (separation-of-duties control)

3A.8 Acceptance Criteria

- AC-RP-01: A new user who registers receives role = 'passenger' in their JWT; no role field is accepted from client input
- AC-RP-02: PATCH /api/v1/admin/users/:id/role by a non-admin returns HTTP 403; by an admin with an invalid role returns HTTP 400; by an admin on their own user_id returns HTTP 403
- AC-RP-03: After successful promotion, a role_profiles record exists with profile_status = INCOMPLETE and the correct promoted_at timestamp
- AC-RP-04: The promoted user's next GET /api/v1/auth/me response includes { profile_completion_required: true, incomplete_role: '<role>' }
- AC-RP-05: A promoted user who has not completed their profile receives HTTP 403 when calling any role-specific endpoint
- AC-RP-06: PATCH /api/v1/profile/complete with missing required fields returns HTTP 422 with a field-level error object
- AC-RP-07: After valid profile submission for a non-verification role, profile_status = COMPLETE and the user can immediately access role features
- AC-RP-08: After valid profile submission for a verification-required role, profile_status = PENDING_VERIFICATION and the user still cannot access role-specific features
- AC-RP-09: After admin approval, profile_status = VERIFIED and the user gains role-specific access on their next request
- AC-RP-10: After admin rejection with a reason, profile_status = INCOMPLETE; the rejection_reason is visible to the user on their next login
- AC-RP-11: Every profile status transition (INCOMPLETE → PENDING_VERIFICATION, PENDING_VERIFICATION → VERIFIED, etc.) is recorded in audit_logs with full before/after state
- AC-RP-12: Adding a new role type by inserting a schema definition does not require changes to the workflow engine; the new role's profile form renders correctly

3A.9 Database Entities and Schema

The following database additions support the Role Upgrade and Profile Completion Workflow:

New Table: role_profiles

Column	Type	Description
id	UUID PK	Unique profile record identifier
user_id	UUID FK → users.id	The user this profile belongs to; unique per role per user
role	ENUM (role_type)	The role this profile record corresponds to
profile_status	ENUM: INCOMPLETE, PENDING_VERIFICATION, COMPLETE, VERIFIED	Current profile completion state; transitions enforced by workflow engine
profile_data	JSONB	Submitted role-specific field values, validated against role_profile_schemas
draft_data	JSONB NULL	Partial/unsaved profile data; allows resuming an incomplete submission
rejection_reason	TEXT NULL	Populated by admin on profile rejection; surfaced to user on next login
promoted_at	TIMESTAMPTZ	Timestamp when admin performed the role promotion
submitted_at	TIMESTAMPTZ NULL	Timestamp when user submitted the profile form
verified_at	TIMESTAMPTZ NULL	Timestamp when admin approved or rejected the profile
verifier_admin_id	UUID FK → users.id NULL	System Administrator who performed the verification decision
created_at	TIMESTAMPTZ	Record creation timestamp
updated_at	TIMESTAMPTZ	Last modification timestamp

New Table: role_profile_schemas

Column	Type	Description
id	UUID PK	Unique schema identifier
role	ENUM (role_type) UNIQUE	The role this schema applies to; one schema per role

requires_verification	BOOLEAN	If true, profile transitions to PENDING_VERIFICATION on submission; admin approval required
field_definitions	JSONB	Array of field definitions: { name, label, type, required, validation_regex, max_length }
version	INTEGER	Schema version; incremented on field changes to support migration of existing profiles
created_at	TIMESTAMPTZ	Record creation timestamp
updated_at	TIMESTAMPTZ	Last modification timestamp

Updated Table: users

The following columns are added or clarified on the existing users table:

Column	Type	Description
role	ENUM: passenger, driver, system_admin, traffic_authority, garage_manager, fuel_station_manager	System-assigned role; DEFAULT 'passenger'; NOT NULL; never accepted from client input
last_role_changed_at	TIMESTAMPTZ NULL	Timestamp of most recent role change; updated by role promotion endpoint

3A.10 Frontend Behavior Specification

Profile Completion Wizard

- Rendered as a full-screen modal or dedicated route (/profile/complete) that intercepts all navigation until profile_status = COMPLETE or VERIFIED
- Fields are rendered dynamically based on the response from GET /api/v1/profile/schema; no hardcoded field lists in the frontend
- Inline validation is applied on blur using the same regex patterns returned by the schema endpoint
- Partial progress is auto-saved every 30 seconds and on field blur
- A progress indicator shows how many required fields remain
- On successful submission, the UI displays a confirmation message and, for verification-required roles, an explanation that approval is pending

Login Flow

- After OTP verification, the frontend checks GET /api/v1/auth/me for profile_completion_required
- If true, the user is redirected to /profile/complete regardless of the originally requested route

- Role-specific navigation items are hidden (not merely disabled) until profile_status = COMPLETE or VERIFIED
- Upon verification approval, the next login or token refresh triggers the full role-specific UI without any further gates

Admin Dashboard — User Management

- User list displays: name, phone, role, profile_status, promoted_at, and action buttons (Promote, View Profile, Verify/Reject)
- Pending Verification tab shows only users with profile_status = PENDING_VERIFICATION and the submitted profile data
- Admin confirmation dialog is required before role promotion or profile verification actions
- Rejection reason field is mandatory in the rejection dialog; submit is blocked if empty

4. Solution Design

4.1 System Architecture

The system follows a layered, modular monolith architecture. All functional domains (authentication, booking, payments, tracking, content, role management) reside within a single deployable Node.js application, organized into clearly separated modules. This approach simplifies deployment, debugging, and team coordination while preserving the option to extract individual modules into separate services in the future if justified by operational demand.

Layer	Component & Responsibility
Client Layer	React.js SPA — role-specific dashboards, real-time tracking UI, booking flows, profile completion wizard
API Layer	Node.js + Express.js REST API — stateless, JWT-authenticated, rate-limited, versioned; RBAC middleware enforces profile_status check on all role-specific routes
Real-Time Layer	Socket.IO — GPS broadcasting, trip rooms, live seat availability updates
Cache Layer	Redis — OTP store, seat reservation locks (TTL), pub/sub bus, session invalidation, profile completion draft data (optional)
Database Layer	PostgreSQL — relational data, transactional seat locking (FOR UPDATE), audit log, role_profiles, role_profile_schemas
Background Jobs	Bull (Redis-backed queue) + Node-cron — booking expiration, retry logic, cleanup jobs, role promotion notification jobs
Payment Gateway	Stripe / Chapa — payment session creation, webhook delivery, refund management
Monitoring	Winston/Pino logging + Sentry + uptime monitoring — structured logs, error tracking, alerting

4.2 Booking & Payment State Machine

All booking status transitions follow a strict finite state machine. The state machine is enforced server-side only; client-supplied state transitions are rejected.

State	Trigger	Next State(s)
RESERVED	Passenger requests booking; seat lock acquired; payment session initialized	PAYMENT_PENDING EXPIRED
PAYMENT_PENDING	Passenger redirected to gateway checkout; awaiting webhook	CONFIRMED FAILED EXPIRED
CONFIRMED	Payment gateway webhook received and verified as successful	Terminal state
FAILED	Payment gateway webhook received and verified as failed/declined	Terminal state; seat released

EXPIRED	Background job detects timeout (15 min) with no completed payment	Terminal state; seat released
---------	---	-------------------------------

4.3 Concurrency Control

Primary: PostgreSQL Row-Level Locking

The primary mechanism for preventing double-booking is a PostgreSQL transaction-level row lock (`SELECT ... FOR UPDATE`) acquired during booking creation. This lock is held within the transaction that creates the booking record, ensuring that concurrent requests for the same seat are serialized at the database level. If the lock cannot be acquired, the transaction returns HTTP 409 Conflict.

Optional: Redis TTL Lock

A Redis TTL lock (key: `lock:trip:{id}:seat:{n}`) may be used as a fast pre-validation step before the database transaction, reducing unnecessary load on the DB under high concurrency. This lock is not required for correctness; the PostgreSQL lock is the authoritative guard.

4.4 Webhook Processing

The payment webhook endpoint (`POST /payments/webhook`) handles asynchronous payment confirmation from the gateway. The processing sequence is:

- Receive incoming webhook request
- Verify HMAC signature using the gateway-provided webhook secret
- Check idempotency key against Redis; reject duplicate events with HTTP 200
- Store idempotency key in Redis with 24-hour TTL
- Update booking status based on payment event type (succeeded / failed)
- Release seat lock if payment failed or expired
- Enqueue notification delivery job for passenger
- Write audit log entry

4.5 Database Design

Core Tables

Table	Key Columns	Notes
users	id, role, phone, name, last_role_changed_at, created_at	role: ENUM('passenger','driver','system_admin','traffic_authority','garage_ma DEFAULT 'passenger'; NOT NULL; system-assigned only
role_profiles	id, user_id, role, profile_status, profile_data (JSONB),	NEW TABLE. Stores profile completion state and submitted data per us INCOMPLETE PENDING_VERIFICATION COMPLETE VERIFIED

	draft_data, rejection_reason, promoted_at, submitted_at, verified_at, verifier_admin_id	
role_profile_schemas	id, role, requires_verification, field_definitions (JSONB), version	NEW TABLE. Defines required fields per role; drives frontend wizard validation. One record per role.
otp_codes	user_id, code, expires_at, attempts	OTPs primarily stored in Redis; DB record for audit trail only
buses	id, plate, capacity, driver_id	One active trip constraint enforced at application and DB level
trips	id, bus_id, status, scheduled_start_time	status: scheduled active completed
bus_locations	trip_id, latitude, longitude, recorded_at	Upsert pattern; one row per trip (latest position only)
bookings	id, passenger_id, trip_id, seat_no, status, expires_at, idempotency_key	status: reserved payment_pending confirmed failed expired
payments	id, booking_id, gateway_txn_id, gateway_status, webhook_event_id, idempotency_key, created_at	No screenshot_url; no admin_id; gateway_txn_id from webhook payload
audit_logs	id, actor_id, action, entity, entity_id, metadata, timestamp	Immutable append-only log. Actions include: ROLE_PROMOTED, PROFILE_VERIFIED, PROFILE_REJECTED, PROFILE_STATUS_CHANGE
posts	id, author_id, content, created_at	Traffic authority announcements; author must have profile_status = CONFIRMED
comments	id, post_id, author_id, content	User interactions on posts
services	id, type, name, lat, lng	type: garage fuel_station; static dataset, extendable

4.6 Technology Stack

Layer	Technology	Rationale
Frontend	React.js	Component-based SPA; dynamic profile wizard rendered from schema API; role-specific routing with completion gate

Backend	Node.js + Express.js	Event-driven I/O; well-suited for WebSocket + REST hybrid workloads; profile workflow middleware
Real-Time	Socket.IO	Mature WebSocket library with rooms, namespaces, and transport fallback
Database	PostgreSQL	ACID transactions; row-level locking; JSONB for flexible profile data storage; reliable at operational scale
Cache / Lock	Redis	Sub-millisecond operations; TTL for reservations and OTPs; pub/sub for Socket.IO scaling; optional draft profile caching
Auth	JWT + OTP	Stateless session authentication; phone-based MFA; JWT claims include profile_completion_required flag
Validation	Zod / Joi	Schema-first validation; profile field validation driven by role_profile_schemas JSONB definitions
Background Jobs	Bull (Redis-backed)	Persistent queue; promotion notification jobs; profile reminder jobs (Phase 2)
Logging	Winston / Pino	Structured JSON logs; configurable log levels; transport plugins
Error Tracking	Sentry (or equivalent)	Stack traces with context; production error alerting
Payment Gateway	Stripe / Chapa	Automated checkout; webhook-based confirmation; refund API

4.7 API Design Principles

- All endpoints versioned under /api/v1/ prefix
- Idempotency keys required on all mutating payment and booking endpoints
- Consistent error response format: { error: string, code: string, details?: object }
- Timeout handling: all external calls (gateway, third-party) have explicit timeouts
- Rate limiting applied at the middleware layer per endpoint category
- OpenAPI/Swagger specification kept in sync with every endpoint change
- Profile completion endpoints return structured field-level validation errors for HTTP 422 responses

4.8 Security Controls

- HTTPS enforced on all non-localhost environments; HTTP requests redirected
- All secrets (database credentials, Redis URL, payment gateway keys, JWT secret) stored in environment variables or a secrets manager; never in source code
- Webhook signatures verified server-side using HMAC before processing
- Input validated on all endpoints using Zod/Joi; strict reject on unexpected fields
- SQL queries use parameterized statements; ORM-level protections applied

- RBAC permission checks enforced at route middleware level on all protected routes; role is system-assigned (never client-supplied); users cannot modify their own role; role escalation attempts return HTTP 403 Forbidden
- Profile completion gate enforced at middleware: all role-specific routes check `profile_status` before allowing access; incomplete profile returns HTTP 403 with code `PROFILE_INCOMPLETE`
- Separation of duties: an administrator who promoted a user may not verify that user's profile for verification-required roles
- Access logs retained for 90 days minimum; audit logs retained indefinitely
- Dependency security audits run monthly via npm audit or Snyk

4.9 Integrations

- Redis Pub/Sub adapter for Socket.IO scaling across multiple Node.js instances
- Bull queue integrates with Redis for reliable background job processing and retry logic
- Payment gateway SDK (Stripe/Chapa) integrated via a standardized adapter pattern to allow future gateway switching
- Browser Geolocation API used client-side for nearby bus discovery
- Webhook endpoint (POST /payments/webhook) receives asynchronous payment confirmation signals
- Role profile schema definitions stored in `role_profile_schemas` table; schema API serves field definitions to the frontend dynamically, eliminating hardcoded role-field mapping in the client

5. Project Organization

5.1 Stakeholders

Stakeholder	Role in Project	Interest / Concern
Project Sponsor	Funds and approves project	ROI, timeline, business outcomes
Project Manager	Owens delivery and governance	Schedule, budget, risk, stakeholder alignment
Passengers	End users; primary beneficiaries	Ease of booking, fair seat access, tracking reliability
Drivers	End users; operational role	Simple trip start/end, location sharing reliability, smooth profile onboarding
System / Operations Administrator	Internal operations role; manages users, roles, verifications, payments, audit access	Operational visibility, role promotion workflow, exception handling, system health
Traffic Authority	End users; information providers	Post management, announcement reach; profile completion on promotion
Garage Manager	End users; facility management role	Facility visibility, operational access after profile completion
Fuel Station Manager	End users; fuel services role	Station visibility, operational access after profile completion
IT / Ops Team	Owens infrastructure and deployment	Stability, monitoring, incident response

5.2 Team Roles and Responsibilities

Role	Responsibility	Deliverable Ownership
Project Manager	Planning, coordination, risk management, stakeholder reporting	Project schedule, status reports, risk register
Backend Lead	API design, database architecture, payment integration, background jobs, security, role profile workflow engine	API docs, schema, auth system, payment webhook, job queue, profile completion API
Frontend Lead	UI/UX implementation, WebSocket integration, state management, profile completion wizard	React components, routing, real-time UI, dynamic profile form
DevOps Engineer	CI/CD, environment management, deployment, monitoring	Pipeline config, server provisioning, log dashboards
QA Engineer	Test planning, execution, regression, performance testing, role workflow testing	Test plan, test cases, bug reports, role upgrade test scenarios

Business Analyst	Requirements gathering, stakeholder communication, UAT support	SRS, user stories, UAT sign-off
------------------	--	---------------------------------

6. Work Breakdown Structure (WBS)

6.1 Major Phases

#	Phase	Description
P1	Planning & Initiation	Scope definition, stakeholder alignment, feasibility analysis
P2	Requirements Analysis	Functional/non-functional requirements, SRS documentation including Role Upgrade Workflow specification
P3	System Design	Architecture, database schema (incl. role_profiles, role_profile_schemas), API contracts, UI wireframes incl. profile wizard
P4	Implementation	Backend, frontend, real-time, background jobs, payment integration, role upgrade workflow
P5	Testing & QA	Unit, integration, system, UAT, performance, security testing; role upgrade workflow test cases
P6	Deployment	Staging, production release, go-live monitoring
P7	Post-Launch Stabilization	Bug triage, performance tuning, operational hand-off

6.2 Tasks and Deliverables

Phase 3: System Design (key additions)

- Database schema: role_profiles and role_profile_schemas tables; updated users table; ERD updated
- API endpoint contracts for all profile completion and admin verification endpoints
- UI wireframes for the Profile Completion Wizard and admin Pending Profiles view
- Profile status state machine diagram and role schema definition format

Phase 4: Implementation (key additions)

- Role Upgrade Workflow: promotion endpoint, profile_status tracking, profile schema API
- Profile Completion API: PATCH /profile/complete, field validation engine, schema-driven validation
- Admin Verification Interface: pending profiles list, approve/reject endpoint with rejection reason
- RBAC middleware updated: profile_status gate on all role-specific routes
- Frontend: Profile Completion Wizard with dynamic field rendering, inline validation, progress indicator, partial save
- Frontend: Admin User Management view with promote, view profile, and verify/reject actions
- Background Jobs: role promotion notification job (Phase 2 notification delivery)

- Audit logging extended for all profile status transitions

Phase 5: Testing & QA (key additions)

- Unit tests: profile schema validation engine, profile status state machine transitions
- Integration tests: all role promotion and profile completion API endpoints
- End-to-end tests: full promotion → incomplete login → profile completion → verification → access flow
- Security tests: bypass profile completion gate, unauthorized role promotion attempts
- UAT: representative user from each new role completes the profile flow in staging

7. Project Schedule

7.1 Milestones

M#	Milestone	Target	Owner
M1	Project Charter & Scope Approved	Week 1	PM
M2	SRS and User Stories Signed Off (incl. Role Upgrade Workflow)	Week 3	BA
M3	Architecture, ERD, API Contracts & Profile Completion Design Approved	Week 5	Tech Lead
M4	Authentication, Booking Core & Role Upgrade Workflow Complete	Week 9	Backend Lead
M5	Payment Gateway Integration & Webhook Processing Complete	Week 11	Backend Lead
M6	Full Backend Feature Complete (incl. Profile Completion API, Admin Verification)	Week 13	Backend Lead
M7	Frontend Development Complete (incl. Profile Wizard, Admin User Management)	Week 16	Frontend Lead
M8	QA Cycle Complete & UAT Sign-Off (incl. Role Workflow scenarios)	Week 19	QA / BA
M9	Staging Deployment & Acceptance	Week 20	DevOps
M10	Production Go-Live	Week 22	PM / DevOps

7.2 Timeline

Phase	Duration	Weeks
Planning & Initiation	2 weeks	Weeks 1–2
Requirements Analysis	2 weeks	Weeks 3–4
System Design (incl. Role Profile Schema Design)	2 weeks	Weeks 5–6
Implementation (incl. Role Upgrade Workflow)	10 weeks	Weeks 7–16
Testing & QA (incl. Role Workflow Test Cases)	3 weeks	Weeks 17–19
Deployment	2 weeks	Weeks 20–21
Post-Launch Stabilization	1 week	Week 22

7.3 Dependencies

- Architecture approval (M3) must precede implementation start, including sign-off on role_profiles schema design
- Authentication system must be complete before booking, payment, and role upgrade modules begin
- Database schema must be finalized (including role_profiles and role_profile_schemas) before any backend service development
- Role profile schema definitions must be seeded before frontend profile wizard can be developed against live API
- Payment gateway credentials and webhook URL must be configured before payment integration testing
- Backend feature complete (M6) is a hard dependency for frontend integration testing
- Full system integration must precede the QA testing phase
- Staging deployment and acceptance (M9) is required before production go-live
- Redis and PostgreSQL infrastructure provisioning must be completed before Week 7

8. Development Approach

8.1 Methodology

The project adopts an Agile-Scrum methodology with 2-week sprints. This allows iterative delivery of working software, early detection of integration issues, and continuous alignment with stakeholder expectations. A design-first phase (Weeks 1–6) establishes firm architectural, schema, and payment integration foundations before agile development sprints begin.

8.2 Development Process

- Sprint planning sessions at the start of each 2-week cycle
- Daily standups (15 minutes) to surface blockers early
- Code reviewed by at least one senior developer before merge
- Feature branches with pull-request workflow; no direct commits to main
- Continuous Integration (CI) pipeline runs linting, unit tests, and build on every pull request
- Sprint retrospectives to identify and act on process improvements
- Backlog grooming every sprint to refine and reprioritize upcoming work

8.3 Quality Standards

- Minimum 80% unit test coverage on all service and controller layers
- All API endpoints must have corresponding integration tests, including all role upgrade and profile completion endpoints
- No merge allowed with a failing CI pipeline or unresolved critical bugs
- OpenAPI/Swagger documentation kept in sync with every endpoint change
- Semantic versioning (SemVer) for all releases
- Security review checklist completed before each staging deployment
- Code linting enforced via ESLint with a team-agreed ruleset

9. Testing Strategy

9.1 Testing Types

Test Type	Scope	Tool / Method
Unit Testing	Individual functions, service methods, validators, profile schema validation engine	Jest / Mocha
Integration Testing	API endpoints, DB queries, Redis interactions, role upgrade workflow API	Supertest + isolated test DB
End-to-End Testing	Full booking, payment webhook, tracking, and role promotion-to-access workflows	Playwright / Cypress
Concurrency Testing	Double-booking attempts under simultaneous load	Custom scripts + k6
Webhook Testing	Payment webhook processing, signature verification, idempotency	Mocked gateway + Stripe CLI / Chapa sandbox
Role Workflow Testing	Promotion, profile completion, pending verification gate, admin verification, rejection/resubmission cycle	Playwright end-to-end + API integration tests
Performance Testing	WebSocket throughput, API response under load	k6 / Artillery
Security Testing	Rate limiting, auth bypass, injection, webhook spoofing, profile gate bypass attempts, unauthorized role promotion	OWASP checklist + manual testing
Regression Testing	All critical paths re-tested after each change	Automated CI suite
UAT	Stakeholder validation of business scenarios including role upgrade and profile completion	Test scripts + formal sign-off

9.2 Acceptance Criteria

Core System

- All booking state transitions execute correctly per the defined state machine
- Concurrent booking attempts for the same seat result in exactly one success and one HTTP 409 Conflict
- Booking auto-expires within 15 minutes of creation when no payment is completed
- GPS updates appear in the passenger UI within 3 seconds of driver emission
- Payment gateway webhook correctly transitions booking status without frontend intervention
- Duplicate webhook events return HTTP 200 and produce no state change (idempotent)

- Webhook with invalid HMAC signature returns HTTP 400 and is not processed
- Rate limiting returns HTTP 429 after threshold is exceeded
- All critical actions appear in the audit log within 1 second of occurrence
- System handles 100 concurrent WebSocket connections without degradation
- UAT sign-off obtained from at least one representative of each stakeholder role

Role Upgrade and Profile Completion

- New user registration assigns role = 'passenger' automatically; no role field accepted from client; verified by inspecting JWT payload and users table after OTP verification (AC-RP-01)
- PATCH /api/v1/admin/users/:id/role called by a non-system_admin returns HTTP 403; called with an invalid role value returns HTTP 400; called by a system_admin targeting their own user_id returns HTTP 403 (AC-RP-02)
- After successful promotion, a role_profiles record exists with profile_status = INCOMPLETE and correct promoted_at timestamp (AC-RP-03)
- Promoted user's next GET /api/v1/auth/me returns { profile_completion_required: true, incomplete_role: '<role>' } (AC-RP-04)
- A promoted user with an incomplete profile receives HTTP 403 with code PROFILE_INCOMPLETE when calling any role-specific endpoint (AC-RP-05)
- PATCH /api/v1/profile/complete with missing required fields returns HTTP 422 with field-level error object (AC-RP-06)
- After valid profile submission for a non-verification role, profile_status = COMPLETE and user immediately accesses role features (AC-RP-07)
- After valid profile submission for a verification-required role, profile_status = PENDING_VERIFICATION and user still cannot access role-specific endpoints (AC-RP-08)
- After admin approval via PATCH /api/v1/admin/users/:id/profile/verify, profile_status = VERIFIED and user gains role-specific access (AC-RP-09)
- After admin rejection with a reason, profile_status = INCOMPLETE and rejection_reason is visible to user on next GET /api/v1/auth/me (AC-RP-10)
- Every profile status transition is recorded in audit_logs with full before/after state, actor_id, and timestamp (AC-RP-11)
- Adding a new role type by inserting a role_profile_schemas record renders the correct profile wizard and validates the correct fields without code changes (AC-RP-12)

RBAC Hardening

- Each role change audit log entry contains actor_id, target_user_id, previous_role, new_role, and timestamp; entries are verified as append-only
- A system_admin attempting to update their own role is rejected with HTTP 403
- A passenger attempting to call the role promotion endpoint is rejected with HTTP 403
- An admin who promoted a user cannot verify that user's profile (separation of duties) — returns HTTP 403 with code CONFLICT_OF_INTEREST

10. Risk Management

10.1 Risk Register

Risk	Likelihood	Impact	Mitigation Plan
Double booking under concurrency	Medium	High	PostgreSQL FOR UPDATE lock (primary); Redis TTL lock (optional pre-validation)
Payment webhook delivery failure	Medium	High	Bull queue retry with exponential backoff (up to 3 attempts); webhook secret rotation policy
Webhook spoofing / replay attack	Low	High	HMAC signature verification on all webhook receipts; idempotency key deduplication
GPS inaccuracy / signal loss	High	Medium	Configurable update intervals; stale-data indicators in UI; graceful degradation to last known position
OTP abuse / brute force	Medium	High	Rate limiting (5/hr/user); attempt caps (3 per OTP); account lockout after repeated failures
WebSocket scaling bottleneck	Low	High	Redis adapter for Socket.IO; stateless backend; horizontal scaling capability
Expired reservations not released	Low	High	Bull queue with persistent jobs; dead-letter queue; monitoring alert on job failure rate
Scope creep from stakeholders	High	Medium	Formal change request process; PM sign-off required for all scope changes
Infrastructure downtime	Low	High	Staged deployment; tested rollback plan; monitoring with automated alerts
Data breach / unauthorized access	Low	Critical	JWT auth, RBAC, input validation, HTTPS enforcement, no plaintext PII in logs, secrets manager usage
Unauthorized role escalation	Low	Critical	Role is system-assigned only; PATCH /api/v1/admin/users/:id/role restricted to system_admin; self-role-change blocked; all changes logged to immutable audit_logs
Profile completion gate bypass	Low	High	Profile_status check enforced server-side at RBAC middleware on all role-specific routes; frontend restrictions alone are not relied upon; penetration-tested
Admin verifies own profile or profile they promoted	Low	Medium	Separation-of-duties control: middleware rejects verification by promoting admin; separate admin must verify; logged in audit_logs
Role schema drift — new role added without schema	Medium	Medium	role_profile_schemas enforces a schema per role at DB level; backend rejects promotion to roles without a defined schema; monitored in CI

Profile data inconsistency — user holds incomplete JSONB profile data	Low	Medium	JSONB validated against role schema on submission; draft_data stored separately from committed profile_data; schema version field enables migration
---	-----	--------	--

11. Budget and Resources

11.1 Resource Requirements

Resource	Count	Responsibility
Project Manager	1	Project delivery, stakeholder communication, governance
Backend Developer	2	API, DB schema, auth, booking, payment gateway integration, background jobs, role upgrade workflow engine
Frontend Developer	1	React UI, WebSocket integration, responsive design, profile completion wizard
DevOps Engineer	1	CI/CD pipeline, environments, monitoring, deployment
QA Engineer	1	Test planning, automation, regression, UAT coordination, role workflow test coverage
Business Analyst	1	Requirements, user stories, UAT support

11.2 Cost Estimates (Monthly Infrastructure)

Category	Description	Estimate (USD/month)
Cloud Infrastructure	Compute (1–2 app servers), managed DB (with JSONB support), Redis, load balancer	\$150 – \$400
Development Tools	CI/CD platform, monitoring (Datadog / Sentry), error tracking	\$50 – \$150
Payment Gateway	Transaction fees: typically 1.5–2.9% + fixed per transaction; no flat monthly fee	Variable (per transaction)
Domain & SSL	Domain registration, TLS certificate (or free via Let's Encrypt)	\$10 – \$50/yr
Personnel	Development team across 22-week delivery horizon	Project-specific budget

12. Deployment Plan

12.1 Environment Strategy

All three environments are strictly isolated with separate configurations, secrets, and databases:

- Development (dev): Local developer machines; hot reload enabled; verbose logging; payment gateway sandbox keys; role_profile_schemas seeded with all role definitions
- Staging: Production-equivalent infrastructure; used for QA, UAT, and pre-release validation; payment gateway test mode; full role upgrade workflow testable end-to-end
- Production (prod): Live environment; hardened security; minimal logging verbosity; payment gateway live keys; monitoring active

12.2 CI/CD Pipeline

- Code push triggers automated lint, unit test, and build steps via GitHub Actions / GitLab CI
- Passing pipeline on the main branch triggers automatic deployment to staging
- Production deployments require a manual promotion gate with PM approval
- Database migrations run automatically as part of the deployment script (Flyway or Knex migrations), including role_profiles and role_profile_schemas table creation and role schema seed data
- Smoke tests execute automatically post-deployment to verify core endpoints, payment webhook reachability, and role schema API availability

12.3 Zero-Downtime Deployment

- Blue/Green or rolling deployment strategy for production releases
- Database migrations designed to be backward-compatible using the expand-contract pattern
- Redis keys versioned to prevent cache poisoning during upgrades
- WebSocket connections gracefully drained before instance restart

12.4 Rollback Plan

- Every production release tagged in version control (SemVer)
- Previous Docker image retained for a minimum of 3 releases; rollback achievable in under 5 minutes
- Database rollback scripts prepared and validated for each migration before deployment, including role_profiles schema changes
- Rollback decision authority: Project Manager in coordination with Tech Lead
- Post-rollback incident report required within 24 hours

12.5 Reliability & Disaster Recovery

Concern	Target / Approach
RTO (Recovery Time Objective)	< 4 hours for P1 incidents
RPO (Recovery Point Objective)	< 1 hour data loss (daily automated DB backups minimum; point-in-time recovery preferred)
Database Backups	Automated daily backups; retained for 30 days minimum; includes role_profiles and role_profile_schemas data
Disaster Recovery Drill	Bi-annual recovery drill to validate backup restoration and failover procedures
Health Checks	Liveness and readiness endpoints (/health) polled by load balancer; automated restart on failure
Graceful Degradation	GPS tracking degrades gracefully if Redis pub/sub is unavailable; REST fallback enabled; profile completion flow remains available without Redis

13. Operations and Maintenance

13.1 Support Plan

Priority	Definition	Response / Resolution SLA
P1 – Critical	System down or data loss	Respond: 1 hr Resolve: 4 hr
P2 – High	Core feature broken (booking/payment/tracking/role workflow)	Respond: 4 hr Resolve: 24 hr
P3 – Medium	Feature degraded; workaround exists; profile completion UX issues	Respond: 1 day Resolve: 3 days
P4 – Low	Minor UI issues, cosmetic bugs	Respond: 3 days Resolve: next sprint

13.2 Monitoring and Observability

System Monitoring

- Real-time uptime monitoring with automated alerting (PagerDuty / Slack webhook)
- Server resource dashboards: CPU, memory, disk, network I/O
- Redis memory and connection pool monitoring
- PostgreSQL query performance and slow-query logging
- Background job queue depth and failure rate tracking
- Payment webhook delivery success rate and latency monitoring
- Profile completion funnel monitoring: promotions initiated, profiles submitted, pending verification queue depth, verification approvals/rejections

Application Monitoring

- Centralized structured logging via Winston/Pino with log aggregation (Datadog / ELK)
- Error tracking with stack traces and context (Sentry or equivalent)
- API endpoint latency and error rate dashboards
- WebSocket connection count and event throughput metrics

Audit Logging

An immutable audit log captures all critical system actions with actor identity, timestamp, entity affected, and action metadata. This includes:

- All booking state transitions (created, expired, confirmed, failed, cancelled)
- All payment events (session created, webhook received, status transitions)
- All system administrator actions (refunds initiated, disputes raised, manual overrides)
- All authentication events (OTP sent, login success, login failure, logout)

- All role management events: ROLE_PROMOTED (actor_id, target_user_id, previous_role, new_role, timestamp), PROFILE_SUBMITTED (user_id, role, submitted_at), PROFILE_STATUS_CHANGED (user_id, role, from_status, to_status, actor_id, timestamp), PROFILE_REJECTED (user_id, role, verifier_admin_id, rejection_reason, timestamp)

Audit logs are append-only and accessible only to authorized System Administrators. Log retention: indefinite for audit records; 90 days for access logs.

Scheduled Maintenance

- Monthly dependency security audits (npm audit / Snyk)
- Quarterly performance review and capacity planning
- Database index review and vacuum schedule
- Bi-annual disaster recovery drill
- Quarterly review of role_profile_schemas to ensure field definitions remain current and accurate

14. Project Governance

14.1 Communication Plan

Communication	Audience	Frequency	Owner
Daily Standup	Dev Team	Daily (15 min)	Scrum Master
Sprint Review	Team + Stakeholders	Bi-weekly	PM
Status Report	Sponsor + PM	Weekly	PM
Risk Review	PM + Tech Lead	Bi-weekly	PM
Steering Committee	Sponsor + PM + Leads	Monthly	PM
Incident Report	PM + Ops + Stakeholders	On incident	DevOps / PM

14.2 Change Management

All proposed scope, schedule, or budget changes must follow the formal change request process:

- Requestor submits a Change Request Form (CRF) with impact description
- Tech Lead assesses technical feasibility and effort estimate
- PM assesses schedule and budget impact
- Steering Committee reviews and approves or rejects
- Approved changes are documented and reflected in the project plan
- Rejected changes are logged with rationale for future reference

Changes classified as Critical (affecting go-live date or core architecture) require sponsor-level approval.

14.3 Reporting Process

- Weekly status report covers: completed work, upcoming work, blockers, risks, and budget status
- Sprint velocity tracked and reported at each sprint review
- Defect metrics (open, resolved, re-opened) reported weekly during testing phase
- Post-launch KPI dashboard shared with sponsor monthly: uptime, bookings, payment success rate, errors, role promotion funnel metrics

15. Appendices

Appendix A: System Architecture Diagram

Layer	Components
Client (Browser)	React.js SPA — HTTPS / WSS; includes Profile Completion Wizard and Admin User Management
API Server	Node.js + Express.js REST API Socket.IO Real-Time Server RBAC Middleware with profile_status gate
Cache & Pub/Sub	Redis — OTP store, seat locks (TTL), Socket.IO pub/sub, session invalidation
Database	PostgreSQL — transactional data, row-level locking, audit log, role_profiles, role_profile_schemas
Background Jobs	Bull Queue + Node-cron — booking expiry, webhook retries, notifications, role promotion notification jobs
Payment Gateway	Stripe / Chapa — hosted checkout, webhook delivery, refund API
Observability	Winston/Pino logging Sentry error tracking Monitoring dashboard Audit log DB Profile funnel metrics

Appendix B: Entity Relationship Diagram (ERD)

Entity		Relationship
users	↔	One user can have many bookings, posts, comments, and role_profiles records
role_profiles	↔	One record per user per role; references users.id and role_profile_schemas.role; linked to audit_logs
role_profile_schemas	↔	One schema per role; defines field_definitions JSONB array and requires_verification flag; referenced by role_profiles
buses	↔	One bus has many trips; one bus has at most one active trip
trips	↔	One trip has many bookings; one trip has one bus_location record
bookings	↔	One booking has one payment record; belongs to one passenger and one trip
payments	↔	One payment belongs to one booking; populated by gateway webhook
posts	↔	One post belongs to one traffic authority user; one post has many comments
audit_logs	↔	Each log entry references actor (user), entity type, entity ID, action metadata; includes all role and profile status events

Appendix C: API Endpoint Reference

Authentication

- POST /api/v1/auth/send-otp — Send OTP to phone number or email (registration or login)
- POST /api/v1/auth/verify-otp — Verify OTP and issue JWT; system automatically assigns role = 'passenger' for new registrations; no role field accepted from client
- GET /api/v1/auth/me — Returns current user including role, profile_completion_required, incomplete_role, and rejection_reason if applicable
- POST /api/v1/auth/logout — Invalidate current session (token blacklisted in Redis)

Profile Completion

- GET /api/v1/profile/schema — Returns the field definitions for the user's current role from role_profile_schemas
- PATCH /api/v1/profile/complete — Submit completed role-specific profile; validates fields against role schema; transitions profile_status

Booking

- POST /api/v1/bookings — Create new reservation (seat lock acquired; payment session initialized)
- GET /api/v1/bookings/my — List current user's bookings
- PATCH /api/v1/bookings/:id/cancel — Cancel a reservation (passenger-initiated)

Payment

- POST /api/v1/payments/webhook — Receive payment gateway webhook (HMAC verified; idempotent)
- GET /api/v1/payments/:id — Retrieve payment record [system_admin or booking owner]

Trips

- POST /api/v1/trips — Schedule a new trip [driver role, profile_status = VERIFIED]
- POST /api/v1/trips/:id/start — Start active trip [driver role]
- POST /api/v1/trips/:id/end — Complete active trip [driver role]
- GET /api/v1/trips/scheduled — List upcoming scheduled trips
- GET /api/v1/trips/nearby — Get nearby active trips by passenger geolocation

Admin

- GET /api/v1/admin/users — Paginated list of all users with current role, profile_status, and promoted_at [system_admin]

- PATCH /api/v1/admin/users/:id/role — Assign or update a user's role [system_admin only]; creates role_profiles record with profile_status = INCOMPLETE; logs audit entry; blocks self-modification
- GET /api/v1/admin/users/pending-profiles — Lists users with profile_status = PENDING_VERIFICATION and submitted profile data [system_admin]
- PATCH /api/v1/admin/users/:id/profile/verify — Approve or reject a submitted profile [system_admin]; body: { decision: 'approved' | 'rejected', reason? }; separation-of-duties enforced
- GET /api/v1/admin/metrics — Revenue, bookings, payment success rates, role promotion funnel metrics [system_admin]
- GET /api/v1/admin/audit-logs — Paginated immutable audit log [system_admin]
- GET /api/v1/admin/health — System health and queue status [system_admin]

Appendix D: Background Job System Design

Job	Trigger	Action
Booking Expiration	Delayed job scheduled on booking creation (expires_at)	Transitions PAYMENT_PENDING booking to EXPIRED; releases seat lock
Webhook Retry	Webhook processing failure	Retries payment webhook processing up to 3 times with exponential backoff
Notification Delivery	Booking status change, role promotion, profile verified/rejected	Sends status notification to user (Phase 2: SMS/email/push)
Profile Completion Reminder	Scheduled 24h and 72h after promotion with profile_status = INCOMPLETE (Phase 2)	Sends reminder to user to complete their role profile
Audit Flush	Batch interval (configurable)	Flushes in-memory audit buffer to audit_logs table
Cleanup Job	Daily cron	Archives expired OTP codes, stale bus_locations, and expired Redis keys

Appendix E: Phased Feature Delivery Plan

Phase	Feature	Description
1.0 (MVP)	Automated Payment Gateway	Stripe / Chapa integration; fully automated booking confirmation via webhooks
1.0 (MVP)	Core Booking & Tracking	Seat reservation, GPS real-time tracking, trip lifecycle management

1.0 (MVP)	Role Upgrade & Profile Completion Workflow	Admin promotion, profile completion gate, schema-driven validation, admin verification for applicable roles
1.0 (MVP)	Operational Dashboard	Revenue, booking analytics, audit log, user management with role promotion interface
2.0	Notifications	SMS/email/push notifications for booking, role promotion, profile completion reminders, verification results
2.0	Advanced ETA Engine	Real-time traffic data integration for dynamic ETA calculations
2.0	Enhanced Analytics	Deeper operational reporting, payment reconciliation, role promotion funnel analytics
3.0	Mobile Application	React Native app for Android and iOS with push notifications
3.0	Traffic-Aware Routing	Third-party map API integration for route suggestions
3.0	Multi-City Expansion	Multi-operator/multi-tenant architecture; new role types supported via extensible role_profile_schemas framework

Appendix F: Glossary

Term	Definition
OTP	One-Time Password: a short-lived numeric code sent to the user for phone or email-based authentication
JWT	JSON Web Token: a compact, signed token used for stateless session authentication; includes role and profile_completion_required claims
Redis TTL	Time-To-Live: automatic expiration of Redis keys after a configured duration
FOR UPDATE	PostgreSQL row-level exclusive lock acquired within a transaction to prevent concurrent modification
Idempotency Key	A unique client-provided or server-generated token ensuring that repeating the same operation has no additional effect
Webhook	An HTTP callback sent from the payment gateway to the backend to notify of a payment status event
HMAC	Hash-based Message Authentication Code: used to verify the authenticity and integrity of webhook payloads
Haversine Formula	A mathematical formula to calculate great-circle distance between two GPS coordinates
Bull Queue	A Redis-backed job queue for Node.js enabling persistent, retrievable background task processing

SRS	Software Requirements Specification: a formal document defining all system requirements
UAT	User Acceptance Testing: formal testing by end users to validate that the system meets business needs
SemVer	Semantic Versioning: a versioning convention (MAJOR.MINOR.PATCH) for software releases
WBS	Work Breakdown Structure: a hierarchical decomposition of the total project scope into manageable tasks
RTO	Recovery Time Objective: the maximum acceptable time to restore a system after a failure
RPO	Recovery Point Objective: the maximum acceptable amount of data loss measured in time
RBAC	Role-Based Access Control: a security model that restricts system access based on the user's assigned role
Expand-Contract	A database migration pattern that ensures backward compatibility during schema changes
Modular Monolith	A single deployable application organized into well-defined internal modules with clear boundaries
role_profiles	The database table storing per-user, per-role profile completion state, submitted data, and verification status
role_profile_schemas	The database table defining the required fields and verification requirements for each role; drives the profile wizard and backend validation
profile_status	An ENUM field (INCOMPLETE, PENDING_VERIFICATION, COMPLETE, VERIFIED) that tracks the completion state of a user's role-specific profile
Profile Completion Wizard	The frontend UI component that guides a newly promoted user through submitting their mandatory role-specific profile fields
Strangler Fig	An architectural pattern where new functionality is built alongside old, gradually replacing it; not directly used here but referenced in refactor guidance
Separation of Duties	A security control requiring that the admin who promotes a user cannot also verify that user's submitted profile, reducing insider risk

Major Changes Summary (v2.0 → v3.0)

New: Role Upgrade and Profile Completion Workflow (Section 3A)

The most significant addition in v3.0 is the complete specification of the Role Upgrade and Profile Completion Workflow. This covers the end-to-end lifecycle from administrator-initiated role promotion through mandatory profile completion, optional administrator verification, and finally full feature access. This section was absent from v2.0.

Expanded Role Enum

The users.role ENUM has been expanded from four values (passenger, driver, system_admin, traffic_authority) to six, adding garage_manager and fuel_station_manager. All role references throughout the document have been updated.

New Database Tables

Two new tables are introduced: role_profiles (stores per-user, per-role completion state and submitted data) and role_profile_schemas (defines required fields per role, drives frontend wizard and backend validation). The users table has been updated with last_role_changed_at.

Security Enhancements

The RBAC middleware now enforces a profile_status gate on all role-specific routes. A new separation-of-duties control prevents the promoting administrator from also verifying the submitted profile. The risk register has been updated with three new role-related risks.

API Surface Expansion

Five new API endpoints are introduced: GET /api/v1/profile/schema, PATCH /api/v1/profile/complete, GET /api/v1/admin/users/pending-profiles, and PATCH /api/v1/admin/users/:id/profile/verify. The existing GET /api/v1/auth/me and GET /api/v1/admin/users endpoints are updated to include profile-related fields.

Extensibility Design

The workflow is designed for extensibility: adding a new role requires only inserting a record into role_profile_schemas. No changes to the workflow engine, middleware, or frontend wizard code are required.

Audit Logging Extended

Four new audit event types are defined: ROLE_PROMOTED, PROFILE_SUBMITTED, PROFILE_STATUS_CHANGED, and PROFILE_REJECTED. All events are recorded in the immutable audit_logs table with full before/after state.

End of Document — Web-Based Transportation Management System Project Plan v3.0
Confidential | TMS Project Plan | Prepared by Alemu Chamada